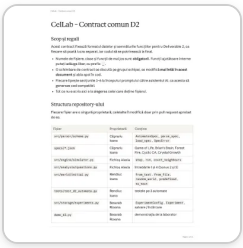
Model pydantic AutomatonSpec și validare

Share



Ajuta-ma sa realizez urmatoarele taskuri ale unui proiect, utilizand informatiile din contextul comun: src/parser/schema.py și spec-urile predefinite.

1. Modelul pydantic v2 AutomatonSpec exact după formatul de mai sus (folosește aliasuri pentru "from"/"if").
2. Validare SEMANTICĂ, nu doar de tip:
   - nume de stări unice, cel puțin 2 stări;
   - "from", "to" și "count" din fiecare regulă sunt stări existente;
   - condiția are exact una dintre formele permise; valorile sunt între 0 și numărul maxim de vecini (8 la moore, 4 la von_neumann); min <= max;
   - probability în [0, 1]; rows și cols > 0; kind, boundary și neighborhood au valori permise;
   - avertisment (nu eroare) dacă o regulă nu poate fi atinsă niciodată pentru că o regulă anterioară cu același "from" o acoperă complet.
3. parse_spec strânge TOATE erorile și aruncă SpecError cu mesaje clare în română care indică regula, de ex. „Regula 2: starea 'ALIV' nu există. Stări cunoscute: DEAD, ALIVE."
4. Scrie în specs/: game_of_life.json, brians_brain.json (OFF/FIRING/DYING, OFF->FIRING dacă exact 2 vecini FIRING), forest_fire.json (determinist: FIRE->EMPTY, TREE->FIRE dacă cel puțin 1 vecin FIRE), cyclic_ca.json (4 stări 0..3, x -> (x+1) mod 4 dacă cel puțin 2 vecini au valoarea următoare), crystal_growth.json (EMPTY->CRYSTAL dacă exact 1 vecin CRYSTAL).
5. tests/test_schema.py: toate spec-urile din specs/ se încarcă fără erori, plus cel puțin 8 spec-uri greșite care produc mesajul așteptat.

Explică-mi pe scurt fiecare decizie de validare, ca să o pot prezenta.

Show less

Scrierea testelor pentru cazurile de configurare invalidă

Scriu fișierele și rulez testele înainte să ți le dau.

Ran 3 commands, shared files

Am scris `schema.py`, cele 5 spec-uri din `specs/` și `test_schema.py`; la `pytest` trec toate cele 49 de teste. Am creat doar fișierele care îți aparțin. `__init__.py` și `pyproject.toml` sunt ale Roxanei, deci le-am făcut doar local ca să pot rula testele.

Cum e structurat

Toată validarea e într-o singură funcție, `_check_raw`, care lucrează pe dicționarul brut și adună toate erorile în română.

- `parse_spec` o apelează înainte de pydantic, ca să primești toate erorile deodată. Dacă ar rula pydantic primul, ar eșua la prima eroare de tip și mesajele ar fi în engleză.
- `AutomatonSpec` o apelează și el într-un `model_validator(mode="after")`, pe `to_dict()`. Un obiect `AutomatonSpec` nu poate exista în stare invalidă, nici dacă un coleg îl construiește direct, fără `parse_spec`.

Deciziile de validare

- **Stări unice, minimum 2, nume** `^[A-Z][A-Z0-9_]*$`: codul unei stări e indexul ei în listă, deci un duplicat ar face codurile ambigue. Regexul din PDF are un typo (`[AZ0-9_]`); l-am citit ca `A-Z`.
- `from`, `to`, `count` **trebuie să fie stări existente:** un nume greșit ar produce mai târziu un `KeyError` în motor, fără să se vadă care regulă e de vină. Mesajul urmează exemplul din cerință: „Regula 2: starea 'ALIV' nu există. Stări cunoscute: DEAD, ALIVE." Pentru `count` mesajul spune „din 'count'".
- **Exact o formă de condiție:** sunt permise `in`, `not_in`, `min`, `max` sau perechea `min` + `max`. Zero forme sau combinații ca `in` + `min` ar avea semantică neclară, așa că le refuz.
- **Valori între 0 și 8 (moore) sau 4 (von_neumann):** o valoare mai mare nu poate fi atinsă niciodată. Verificarea depinde de `neighborhood`, de aceea e într-un singur loc.
- **Liste goale în** `in` / `not_in` **sunt eroare:** `in: []` nu e adevărat niciodată, iar în practică e aproape sigur o greșeală.
- `min <= max`: altfel intervalul e gol.
- `probability` **între 0 și 1:** în afara intervalului comparația cu `rng.random()` n-ar mai avea sens. Booleenii nu sunt acceptați ca numere (`True` e `int` în Python).
- `rows` **și** `cols` **între 1 și 1000:** cerința zicea „> 0", dar contractul fixează și maximul de 1000, așa că le verific pe amândouă.
- `kind`, `boundary`, `neighborhood`, `otherwise` **cu valori permise:** valorile permise sunt `Literal` în model, dar le verific și manual ca mesajul să fie în română.
- **Conflict de simboluri:** verific harta completă, cu simbolurile implicite incluse. De exemplu `FIRING` și `FLAME` ar primi amândouă `F`, iar contractul cere eroare în cazul ăsta.
- **Câmpuri necunoscute sunt eroare (**`extra="forbid"`**):** prinde greșeli ca `neigborhood`, care altfel ar fi ignorate în tăcere.
- `otherwise` **lipsă primește valoarea implicită:** tabelul din contract îl dă „obligatoriu", dar semnătura modelului are default `"unchanged"`. Am urmat semnătura.
- **Avertisment (în** `lint_spec`**, nu eroare):** o regulă e marcată dacă o regulă anterioară cu același `from` o acoperă complet. Asta înseamnă că cea anterioară e necondiționată sau acoperă toate valorile posibile, ori e pe același `count` cu un set de valori care îl include pe al ei.

- O regulă anterioară cu `probability < 1` nu acoperă nimic, pentru că poate să nu se aplice.
- Condițiile pe stări `count` diferite nu se compară, deci nu dau avertisment.
- E avertisment și nu eroare pentru că spec-ul rămâne valid și rulabil.

Alegeri în spec-uri

- `neighborhood` **moore la toate cinci:** contractul nu fixează vecinătatea pentru automatele noi. Am ales moore ca să se potrivească cu ce presupune probabil Rendiuc în testele ei cu grile scrise de mână. Dacă vrei von Neumann la Forest Fire, e o schimbare de o linie, dar anunț-o în grup.
- `boundary: fixed` **la Forest Fire și Crystal Growth:** focul și cristalul sunt fenomene finite, deci nu are sens să se lege marginile. Celelalte trei sunt `toroidal`.
- **Cyclic CA:** am folosit numele `S0` … `S3` cu simbolurile `0` … `3`, ca în exemplul din contract. „0..3” nu e un nume de stare valid, fiindcă numele trebuie să înceapă cu literă.

Download all

38 minutes ago